

MSC IN MATHEMATICAL ENGINEERING

COURSE: ADVANCED PROGRAMMING FOR SCIENTIFIC COMPUTING

Lecturer: Luca Formaggia
Teaching Assistant: Paolo Joseph Baioni

Course project
A.Y. 2024/2025

FlowPDE: Flow-Based Generative Surrogates for Parametrized PDEs

Authors:

Sarper Yurtseven

Mentors:

**Andrea Manzoni
Nicola Farenga
Giacomo Bottacini**

Abstract

This report presents `flowpde`, a PyTorch library that models the conditional distribution $p(\mathbf{u} \mid \boldsymbol{\mu})$ with continuous-time flows. It fits the same conditional neural ODE either by exact maximum likelihood or by flow matching, and its modular design keeps the dynamics, the training objective, the probability path, the time sampler, the coupling and the source distribution as separate components.

Thirteen controlled experiments on Poisson, Burgers and Darcy problems vary conditioning, backbone, training objective, flow-matching components and observation quality, one axis at a time. Removing the conditioning increases the median error by $17\times$, while the loss curve still looks healthy. The smallest of four backbones is the most accurate on every forward problem. On frozen weights, Euler sampling beats fourth-order Runge–Kutta by up to $8.6\times$ at equal cost, and integrating the ODE to convergence is 11% worse than stopping at sixteen Euler steps. Maximum-likelihood training improves test likelihood, worsens sample accuracy, and costs $70\times$ more than flow matching.

For the inverse problems, the learned posterior becomes $3.9\times$ narrower from the lowest to the highest flux decile. This means that the model can identify some parameters better than others, which is also expected from the Darcy problem. These results still have some limitations. Most experiments use only one run per setting, there is no deterministic baseline, and at this problem size the learned method is not faster than the reference solver.

Contents

Abstract	1
1 Introduction	2
1.1 Contributions	3
1.2 Related work	3
2 Generative AI for PDE surrogates	4
2.1 Flow-based generative modeling	4
2.1.1 Continuous normalizing flows	4
2.1.2 Flow matching	5
2.1.3 Rectified flows	7
2.2 Application to parametrized PDEs	8
2.2.1 Forward problems	9
2.2.2 Inverse problems	9
3 FlowPDE	10
3.1 Library overview	10
3.2 Modules and implementation details	11
3.2.1 <code>flowpde.core</code>	11
3.2.2 <code>flowpde.models</code>	12
3.2.3 <code>flowpde.flows</code>	12
3.2.4 <code>flowpde.objectives</code>	12
3.2.5 <code>flowpde.trainers</code>	13
3.2.6 <code>flowpde.datasets</code>	13
3.2.7 <code>flowpde.solvers</code>	14
4 Numerical experiments	14
4.1 Setup	14
4.2 Conditioning, backbone and objective	16
4.3 Accuracy against sampling cost	17
4.4 Inverse problems and calibration	18
4.5 Controlling how ill-posed the problem is	20
4.6 Comparison against MCMC	22
5 Conclusions	23
Bibliography	23

1 Introduction

Solving a partial differential equation once is generally not a difficult problem anymore. However, solving the same problem thousands of times for different inputs can still be very expensive. This is exactly the situation that appears in many applications. For example, design optimization explores many different values in a parameter space. Optimal control requires repeated PDE solves during the optimization process. Uncertainty propagation samples many different inputs from a given distribution. In inverse problems, we search for input parameters such that the corresponding solution matches the given observations. Therefore, in all these problems we are not interested only in one solution. Instead, we are interested in the solution map

$$\boldsymbol{\mu} \mapsto u(\boldsymbol{\mu}),$$

which needs to be evaluated many times.

The main difficulty comes from the computational cost. A fine discretization is generally required to obtain an accurate solution. However, repeating such a high-fidelity solve thousands of times can become too expensive. Reduced order modelling is one classical approach to deal with this problem. The main idea is to split the computation into two parts: an expensive offline stage that is performed once, and a cheap online stage that is used for every new input [25]. Deep learning methods extend this idea in different ways. Nonlinear models can replace the classical linear reduced spaces [9], while neural operators try to learn the solution map directly between function spaces [18, 19]. Most of these methods are deterministic. Meaning that, for each input, the model returns only one solution field. These models are also commonly trained using the mean squared error. The population minimizer of the mean squared error is the conditional mean

$$\mathbb{E}[u \mid \boldsymbol{\mu}].$$

If the solution map is deterministic, this is not a problem. In this case, each input $\boldsymbol{\mu}$ corresponds to only one solution $u(\boldsymbol{\mu})$, and therefore the conditional distribution is concentrated around this solution.

However, this is not always the case. For example, in inverse problems, sparse and noisy observations may correspond to many different coefficient fields. Similarly, partial observations may not uniquely determine the complete solution. In chaotic systems, small differences in the initial condition may also lead to completely different trajectories after sufficiently long time. In these situations, there is not only one possible solution for a given observation. Therefore, using only the conditional mean can remove important information about the possible solutions. This becomes especially problematic for nonlinear equations. If we average several valid solutions of a nonlinear problem, the resulting average is not necessarily a valid solution itself. Therefore, a deterministic model may return a smooth field that does not correspond to any of the actual possible solutions. In this report, instead of predicting only one solution, we model the conditional distribution

$$p(u \mid \boldsymbol{\mu})$$

and sample from this distribution.

In this way, for the same input $\boldsymbol{\mu}$, we can generate several possible solutions. Each sample represents one possible realization, while the variation between samples gives information about the uncertainty of the problem. For this purpose, we use continuous-time flow-based generative models. In particular, we consider continuous normalizing flows [3, 11] and flow matching methods [1, 20, 23]. There are two main properties of these methods that are useful for PDE surrogate modelling.

Sampling is an ODE solve. After training, a sample is generated by solving an ordinary differential equation. Therefore, the number of function evaluations can be changed depending on the desired balance between computational cost and accuracy. This can also be changed after training without modifying the learned model.

Conditional training is amortized. One training procedure gives a sampler for the whole family

$$\{p(\cdot \mid \boldsymbol{\mu})\}_{\boldsymbol{\mu}}.$$

Therefore, once the model is trained, it can be used for many different values of $\boldsymbol{\mu}$. This gives a similar offline/online structure to classical reduced order modelling.

The main contribution of this work is the library `flowpde`, rather than one specific algorithm.

There are many different choices when constructing a flow-based PDE surrogate. These include the interpolation path, time sampling strategy, noise-data coupling, source distribution, neural network backbone, training objective, ODE solver and the number of integration steps.

Usually, different methods are implemented in different codebases. This makes comparisons difficult because changing one component may require implementing another model almost from scratch.

In `flowpde`, these components are separated from each other. Therefore, one component can be changed while the others remain fixed. This makes comparisons between different methods easier and more controlled.

1.1 Contributions

- **The flow is separated from the objective.** The dynamics of the model and the loss used to train them are implemented separately. Therefore, the same conditional neural ODE can be trained using exact maximum likelihood or flow matching without changing the model, dataset or sampling procedure. This allows us to compare simulation-based and simulation-free training under the same setting (Section 3.1).
- **Flow matching is divided into four independent components.** The interpolation path, time sampler, coupling and source distribution can each be changed independently. Therefore, the different variants discussed in Section 2.1.2 are configurations of the same implementation instead of completely different subclasses (Section 3.2). In Section 4.3, six of these configurations are compared under the same experimental setup.
- **Forward and inverse generators with controllable ill-posedness.** Poisson, Burgers and Darcy problems are generated using one spectral pipeline. The problems can be considered in both forward and inverse directions. Observation noise and random masking can also be changed directly. In Section 4.5, these parameters are varied from cases where the posterior contains little additional information compared with the prior to cases where the observations are much more informative.
- **Distributional evaluation instead of only relative L^2 error.** For probabilistic predictions, the relative L^2 error alone is not sufficient. Therefore, we also use proper scoring rules, coverage, rank histograms, spread-skill ratios and variance decomposition. These quantities are reported in physical units when possible (Section 2.2.2).
- **A controlled study of the main design choices.** We vary the conditioning mechanism, backbone, training objective, flow matching components and observation quality. The same data, normalization, optimizer and sampling procedure are used whenever possible. Therefore, each experiment changes only one main component. The cheaper experiments are also repeated using matched random seeds (Section 4).

1.2 Related work

Flow-based generative modelling. Normalizing flows were introduced as generative models based on invertible transformations [26]. Continuous normalizing flows later extended this idea using neural ODEs [3], while FFJORD allowed more flexible continuous dynamics [11]. Diffusion and score-based models then became widely used generative methods [14, 28]. More recently, flow matching [20], rectified flow [23] and stochastic interpolants [1] provided simpler approaches for training continuous-time generative models. Some later developments include minibatch optimal transport couplings [30] and different time sampling strategies [7]. Section 2.1 explains these methods in more detail, while Lipman et al. [21] provides a general treatment of flow matching.

Generative models for PDEs. Generative models have also been applied to PDE problems. PDE-Refiner [22] uses a short denoising procedure to improve the predicted solution and recover low-amplitude spectral information that can be missed when training only with mean squared error. Kohl et al. [17] study autoregressive conditional diffusion models for turbulence. They show that the long-time statistics produced by the probabilistic model can be more accurate than those obtained from deterministic predictors. GenCast [24] applies a similar probabilistic approach at a much larger scale for weather prediction. Most of the existing literature is based on diffusion models rather than flow matching. Moreover, many works focus on time-dependent problems, where uncertainty naturally appears during long-time prediction. Flow matching has been studied less for parametrized elliptic and parabolic PDEs, especially when both forward and inverse problems are considered. Another important point is that probabilistic predictions should not only generate different samples, but the predicted distribution should also be evaluated in terms of calibration and uncertainty. For inverse problems, Wildberger et al. [31] train a conditional flow matching model as an amortized posterior estimator. The inverse problem formulation used in Section 2.2.2 follows the same general idea.

Section 2 reviews flow-based generative modelling and its application to parametrized PDEs. Section 3 describes the `flowpde` library. Section 4 presents the numerical experiments.

2 Generative AI for PDE surrogates

In this section, we introduce the main concepts that will be used throughout the rest of the report. First, Section 2.1 introduces flow-based generative models and three different ways to train them. Then, Section 2.2 connects these models to partial differential equations. In particular, we discuss what distribution we want to model and when using a probabilistic model is useful.

2.1 Flow-based generative modeling

Assume that we have an unknown distribution q on $\mathbb{R}^d$. We only have samples from this distribution, and we want to generate new samples from it. The main idea of flow-based generative models is to move a simple reference distribution $\rho_0 = \mathcal{N}(\mathbf{0}, I)$ to the target distribution q . Let $\mathbf{v}_t(\mathbf{x})$ be a time-dependent velocity field on $[0, 1] \times \mathbb{R}^d$, and let ψ_t be the corresponding flow map, defined by

$$\frac{d}{dt}\psi_t(\mathbf{x}_0) = \mathbf{v}_t(\psi_t(\mathbf{x}_0)), \quad \psi_0(\mathbf{x}_0) = \mathbf{x}_0. \quad (1)$$

If $\mathbf{v}$ is Lipschitz continuous in $\mathbf{x}$, then by the Picard–Lindelöf theorem the solution is unique. Therefore, ψ_t is a diffeomorphism for every t , and its inverse can be obtained by integrating the ODE backwards. This means that invertibility follows directly from the ODE formulation. We do not need to design the neural network itself to be explicitly invertible. This is one of the main advantages of the continuous formulation, as discussed in Section 2.1.1.

Pushing ρ_0 through this flow gives a family of distributions $p_t = (\psi_t)_\# \rho_0$, which we call the *probability path*. Equivalently, this probability path satisfies the continuity equation

$$\partial_t p_t(\mathbf{x}) + \nabla \cdot (p_t(\mathbf{x}) \mathbf{v}_t(\mathbf{x})) = 0, \quad p_0 = \rho_0. \quad (2)$$

We say that the velocity field $\mathbf{v}$ generates the probability path p_t . Therefore, the generative problem is to find a velocity field such that $p_1 = q$. There are many different velocity fields that can satisfy this condition. The methods below mainly differ in the objective that is used to learn this field.

Remark 1 (One object, three training routes). *The next three subsections describe the same basic object: a parameterized velocity field $\mathbf{v}_\theta$ and the probability path generated by it. The main difference between them is the training objective. Section 2.1.1 trains the model by maximizing exact likelihood. This requires solving an ODE inside each gradient step. Section 2.1.2 instead trains the model by regression against a closed-form target velocity. Therefore, no ODE solve is needed during training. Finally, Section 2.1.3 keeps the same regression objective but changes how the source and target samples are paired in order to obtain straighter trajectories. Therefore, continuous normalizing flows, flow matching and rectified flow can be seen as three different training procedures for the same model family. In FlowPDE, they use the same flow class and differ through their training objectives.*

2.1.1 Continuous normalizing flows

A classical normalizing flow [26] is constructed by stacking several invertible transformations. Since every transformation is invertible, the change-of-variables formula can be used to compute the exact density

$$\log p_1(\mathbf{x}) = \log \rho_0(\psi^{-1}(\mathbf{x})) - \sum_{k=1}^K \log \left| \det \frac{\partial \psi^{(k)}}{\partial \mathbf{x}} \right|. \quad (3)$$

This ability to compute the exact density is one of the main differences between normalizing flows and generative models such as variational autoencoders or adversarial networks. However, there is a computational cost. Computing a general determinant costs $\mathcal{O}(d^3)$. Therefore, classical normalizing flows usually require special architectures, such as triangular or coupling-based transformations, for which the determinant can be computed efficiently. This means that the architecture is partially designed according to the Jacobian computation instead of only according to the problem. Chen et al. [3] removed this restriction by introducing continuous normalizing flows. Instead of stacking discrete transformations, we parameterize the velocity field $\mathbf{v}_\theta$ in (1) using a neural network.

Invertibility now follows from the uniqueness of the ODE solution. At the same time, the log-determinant is replaced by the *instantaneous change of variables*

$$\frac{d \log p_t}{dt} = -\text{tr} \left(\frac{\partial \mathbf{v}_t}{\partial \mathbf{x}} \right),$$

along a trajectory. This follows from (2) by expanding the divergence and dividing by p_t . Therefore, the $\mathcal{O}(d^3)$ determinant is replaced by an $\mathcal{O}(d)$ trace, while the sum over discrete transformations becomes an integral:

$$\log p_1(\mathbf{x}_1) = \log \rho_0(\mathbf{x}_0) - \int_0^1 \text{tr} \left(\frac{\partial \mathbf{v}_\theta}{\partial \mathbf{x}}(\mathbf{x}(t), t) \right) dt. \quad (4)$$

Both the state and the log-density can be obtained from one backward solve of the augmented system $[\mathbf{x}, \log p]$. The model can then be trained by minimizing

$$\mathcal{L}_{\text{NLL}}(\boldsymbol{\theta}) = -\mathbb{E}_{\mathbf{x}_1 \sim q} [\log p_1^\theta(\mathbf{x}_1)]. \quad (5)$$

The trace computation still requires d Jacobian-vector products. FFJORD [11] reduces this cost by using the Hutchinson estimator [15]. For $\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$ and $\text{Cov}(\boldsymbol{\varepsilon}) = I$, we have

$$\text{tr}(A) = \mathbb{E}_{\boldsymbol{\varepsilon}} [\boldsymbol{\varepsilon}^\top A \boldsymbol{\varepsilon}], \quad \text{tr} \left(\frac{\partial \mathbf{v}_\theta}{\partial \mathbf{x}} \right) \approx \frac{1}{M} \sum_{m=1}^M \boldsymbol{\varepsilon}_m^\top \left(\frac{\partial \mathbf{v}_\theta}{\partial \mathbf{x}} \boldsymbol{\varepsilon}_m \right). \quad (6)$$

Each term requires one backward pass. The estimator is unbiased for any $M \geq 1$. However, its variance increases with the dimension d , and therefore larger values of M may sometimes be needed. Even after using this estimator, continuous normalizing flows still have some important computational costs. These also motivate the flow matching approach introduced in Section 2.1.2.

Training requires simulation. Each gradient step requires solving an ODE, which can require tens or hundreds of neural network evaluations.

The dynamics can become more difficult during training. The objective in (5) depends on $\mathbf{v}_\theta$ through the final map at $t = 1$. Therefore, it does not directly force the intermediate dynamics to remain simple. As a result, the velocity field can become stiff during training and the cost of the ODE solve can increase [11].

Regularization is usually added separately. Kinetic-energy or Jacobian penalties can be introduced to encourage smoother and straighter trajectories. Flow matching instead obtains a simpler transport path directly from its construction.

Conditioning does not change the main formulation. We simply write $\mathbf{v}_\theta(\mathbf{x}, t; \mathbf{c})$ and replace q with the conditional distribution $q(\cdot | \mathbf{c})$.

2.1.2 Flow matching

Suppose for a moment that we already know a probability path p_t from ρ_0 to q , together with a velocity field $\mathbf{u}_t$ that generates this path. In this case, training $\mathbf{v}_\theta$ would simply be a regression problem and would not require solving an ODE during training [20]. However, in practice, we know neither the probability path nor its velocity field. The main idea of flow matching is to construct the path from simpler conditional paths that we can define analytically. We first sample $\mathbf{x}_1 \sim q$ and define a conditional probability path such that

$$p_0(\cdot | \mathbf{x}_1) = \rho_0, \quad p_1(\cdot | \mathbf{x}_1) \approx \delta_{\mathbf{x}_1}. \quad (7)$$

Therefore, each conditional path transports the reference distribution toward one particular data sample. If we marginalize over $\mathbf{x}_1$, we obtain the probability path

$$p_t(\mathbf{x}) = \int p_t(\mathbf{x} | \mathbf{x}_1) q(\mathbf{x}_1) d\mathbf{x}_1, \quad p_0 = \rho_0, \quad p_1 = q. \quad (8)$$

The conditional velocity field $\mathbf{u}_t(\mathbf{x} | \mathbf{x}_1)$ can be computed analytically. However, the corresponding marginal velocity field is

$$\mathbf{u}_t(\mathbf{x}) = \int \mathbf{u}_t(\mathbf{x} | \mathbf{x}_1) \frac{p_t(\mathbf{x} | \mathbf{x}_1) q(\mathbf{x}_1)}{p_t(\mathbf{x})} d\mathbf{x}_1. \quad (9)$$

This integral is generally not tractable because it requires integrating over all data points that may have generated the current state $\mathbf{x}$. The important point is that we do not need to evaluate this integral directly.

Theorem 1 (Conditional flow matching; 20). *Let p_t and $\mathbf{u}_t$ be the marginal probability path and marginal velocity field defined in (8) and (9). Then $\mathbf{u}_t$ generates p_t , and with*

$$\mathcal{L}_{\text{CFM}}(\boldsymbol{\theta}) = \mathbb{E}_{t, \mathbf{x}_1 \sim q, \mathbf{x} \sim p_t(\cdot | \mathbf{x}_1)} \|\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t) - \mathbf{u}_t(\mathbf{x} | \mathbf{x}_1)\|^2, \quad (10)$$

the gradients of the marginal and conditional objectives are equal. Therefore, the two objectives have the same minimizers with respect to $\boldsymbol{\theta}$.

Sketch of proof. We expand the squared norms in both objectives. The $\|\mathbf{v}_{\boldsymbol{\theta}}\|^2$ terms are the same because $\mathbf{x}$ has the same marginal distribution in both expectations. The terms that do not depend on $\boldsymbol{\theta}$ disappear after taking the gradient. Finally, the cross terms become equal after substituting (9). $\square$

This result means that we can replace the marginal target, which is difficult to compute, with the conditional target, which is available analytically, without changing the minimizer of the objective. There is one important consequence. The model cannot generally fit every conditional target exactly. Different source–target pairs may produce the same point $\mathbf{x}_t$ at the same time t , while having different target velocities. Therefore, the model converges to the conditional average of these velocities, which is exactly the marginal velocity field $\mathbf{u}_t$. This also means that the training residual does not necessarily converge to zero. This is why Section 3.2.5 selects models based on sampled error and not only on the training loss.

Gaussian paths, and diffusion as a special case. A common choice is to use Gaussian conditional paths,

$$p_t(\mathbf{x} | \mathbf{x}_1) = \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_t(\mathbf{x}_1), \sigma_t(\mathbf{x}_1)^2 I),$$

with generating velocity field

$$\mathbf{u}_t(\mathbf{x} | \mathbf{x}_1) = \frac{\sigma'_t}{\sigma_t}(\mathbf{x} - \boldsymbol{\mu}_t) + \boldsymbol{\mu}'_t.$$

Different choices of $\boldsymbol{\mu}_t$ and σ_t give different known continuous-time generative models. For example, the variance-preserving and variance-exploding schedules used in score-based diffusion models [14, 28] can also be written in this framework. Their probability-flow ODE is therefore one specific case of (1) obtained from a particular Gaussian path.

However, these diffusion schedules are not necessarily the simplest choice for flow matching. They come from a noising process that gradually removes information from the data, and this generally produces curved trajectories.

Instead, Lipman et al. [20] use the optimal-transport path

$$\boldsymbol{\mu}_t(\mathbf{x}_1) = t\mathbf{x}_1, \quad \sigma_t = 1 - (1 - \sigma_{\min})t. \quad (11)$$

Here, both the mean and the standard deviation change linearly with time. When $\sigma_{\min} \rightarrow 0$, this reduces to

$$\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1, \quad \mathbf{u}_t(\mathbf{x}_t | \mathbf{x}_0, \mathbf{x}_1) = \mathbf{x}_1 - \mathbf{x}_0. \quad (12)$$

Therefore, the conditional path is simply the straight line from the source sample $\mathbf{x}_0$ to the data sample $\mathbf{x}_1$, with constant velocity. This gives a very simple training target.

Remark 2 (Interpolation and velocity must agree). *In (12), the target velocity is exactly the time derivative of the interpolation.*

Therefore, we must have

$$\mathbf{u}_t = \partial_t \mathbf{x}_t.$$

This is not only a notation choice. It is a consistency condition between the interpolation and the velocity field. If these two do not agree, then the model receives samples from one path while being trained with the velocity of another path. In this case, the learned velocity field does not generate the intended probability path. For this reason, an implementation that supports several interpolation paths should also check that the corresponding velocity is the time derivative of the path. FlowPDE treats this relation as a tested condition.

Couplings. Theorem 1 is usually written using the independent coupling

$$\pi = \rho_0 \otimes q.$$

However, the same result also holds for other couplings π with the correct marginals [1, 30]. The choice of coupling is important because it changes the difficulty of the regression problem. With independent coupling, the target velocity at the same point $(\mathbf{x}_t, t)$ can vary significantly between different samples. This increases the variance of the regression target and therefore increases the minimum possible training loss.

Minibatch optimal transport [30] tries to reduce this variance by choosing better source–target pairs inside each batch.

We solve

$$\sigma^* = \arg \min_{\sigma \in S_B} \sum_{i=1}^B \left\| \mathbf{x}_0^{(\sigma(i))} - \mathbf{x}_1^{(i)} \right\|^2 \quad (13)$$

and train using the reordered pairs. The idea is that pairing closer source and target samples gives shorter interpolation paths and makes the target velocities more similar. However, there are two limitations. First, the minibatch transport plan is a biased approximation of the population optimal transport plan. This bias decreases as the batch size B increases. Second, solving (13) exactly has computational complexity $\mathcal{O}(B^3)$. The stochastic-interpolant formulation [1] starts from the interpolation itself rather than from the probability density. This makes two additional choices explicit. The first is the *source* distribution of $\mathbf{x}_0$. Instead of being treated only as the limit of a noising process, $\mathbf{x}_0$ is considered directly as a random variable. The second is the *coupling*, which determines how source and target samples are paired. Both the source and the coupling are implemented as separate components in FlowPDE, as discussed in Section 3.

Time sampling and conditioning. The standard flow matching derivation uses

$$t \sim \mathcal{U}[0, 1].$$

However, the regression problem is not equally difficult for every value of t . Therefore, different time distributions can be used to sample more often from regions where the regression error is larger. One common choice is the logit-normal schedule of Esser et al. [7]. Conditioning again does not change the main formulation. We write

$$\mathbf{v}_\theta(\mathbf{x}, t; \mathbf{c}),$$

where $\mathbf{c}$ remains fixed while $\mathbf{x}$ evolves according to (1). After training, sampling consists of drawing one source sample and solving (1).

2.1.3 Rectified flows

The computational cost of sampling depends on the number of function evaluations used by the ODE solver. This number is strongly affected by how curved the trajectories are. If a trajectory is a straight line and moves with constant velocity, then one Euler step is enough to integrate it exactly. Therefore, it is useful to measure how straight the learned trajectories actually are.

Definition 1 (Straightness; 23). *For a flow with trajectories $\mathbf{z}_t$, define*

$$S = \int_0^1 \mathbb{E} \left[\left\| (\mathbf{z}_1 - \mathbf{z}_0) - \mathbf{v}_\theta(\mathbf{z}_t, t) \right\|^2 \right] dt. \quad (14)$$

Then $S = 0$ if every trajectory follows the straight line between its initial and final point with constant velocity.

This quantity measures the deviation from the straight line connecting the two endpoints. Therefore, it measures more than only the change in speed. For example, a trajectory can move with constant speed and still have $S > 0$ if its direction changes. At first, one might expect the flow to already be straight because the conditional paths in (12) are straight lines. However, the marginal flow is not necessarily straight.

The reason is that different conditional paths can cross. Two different source–target pairs can pass through the same point $(\mathbf{x}_t, t)$ while having different target velocities. At this point, the marginal velocity field in (9) can only take one value. This value is the conditional average of the possible velocities.

Moreover, trajectories of the same ODE cannot intersect. Therefore, the marginal flow follows a different non-crossing transport and its trajectories can become curved. Hence, straight conditional paths do not necessarily give a straight marginal flow, and in general we can have

$$S > 0.$$

The reflow procedure. Reflow [23] tries to obtain straighter trajectories by changing the coupling between source and target samples. Assume that we already have a trained velocity field $\mathbf{v}_\theta$. We first sample

$$\mathbf{z} \sim \rho_0$$

and integrate (1) accurately to obtain

$$\mathbf{x}'_1 = \text{ODE}(\mathbf{z}).$$

We then retrain the model using the pairs

$$(\mathbf{z}, \mathbf{x}'_1).$$

The important difference is that these pairs are produced by the learned ODE itself. Therefore, this coupling is deterministic and non-crossing. Since the source and target samples are already connected by the learned flow, the same flow matching objective can now learn straighter trajectories between them. Repeating this procedure gives the k -rectified flow. The marginal distributions are preserved at each reflow step, while the transport cost does not increase.

Remark 3 (Rectified flow is not the same thing as reflow). *Rectified flow and reflow are often used as if they were the same thing, but there is an important difference. Training with a linear interpolation and an independent coupling, usually called “1-rectified flow”, is mathematically the same as standard flow matching with a linear path. Therefore, this step by itself does not perform any additional trajectory straightening. Reflow is the retraining procedure where the source samples are passed through the trained model and then paired with the generated target samples. This change in the coupling is what produces the trajectory straightening. FlowPDE keeps rectified flow and reflow as separate concepts (Section 3.2.3).*

Remark 4 (Reflow needs the original pairing). *Reflow requires the original source–target pairing to be preserved during training. Assume that a source sample $\mathbf{z}$ is passed through the trained ODE and produces the target sample $\mathbf{x}'_1$. Then the same source sample $\mathbf{z}$ must be used together with $\mathbf{x}'_1$ during reflow training. If we instead sample new independent noise, the original pairing is lost. In this case, we no longer use the deterministic coupling generated by the previous flow, and the trajectory straightening effect disappears. An important problem is that the training loss may still look normal in this case. Therefore, this type of implementation error may not be visible from the loss curve alone. For this reason, FlowPDE treats the source of $\mathbf{x}_0$ as an explicit component instead of always sampling a new $\mathcal{N}(\mathbf{0}, I)$ inside the loss.*

2.2 Application to parametrized PDEs

So far, we introduced flow-based generative models without discussing partial differential equations. We now connect this framework to parametrized PDEs. There are two main questions. First, what is the distribution that we want to model? Second, in which cases is modelling a distribution actually useful?

The parametrized problem. Let $\boldsymbol{\mu} \in \mathcal{P}$ be a parameter. This parameter may represent physical coefficients, or it may itself be a field, such as a diffusion coefficient, forcing term or initial condition.

We consider the parametrized PDE

$$\mathcal{A}(u; \boldsymbol{\mu}) = 0 \quad \text{in } \Omega, \quad \mathcal{B}(u) = 0 \quad \text{on } \partial\Omega. \quad (15)$$

Assume that the problem is well posed. Then, for every parameter $\boldsymbol{\mu}$, there is one corresponding solution $u(\boldsymbol{\mu})$. Therefore, we can define the solution map

$$\mathcal{S} : \boldsymbol{\mu} \mapsto u(\boldsymbol{\mu}).$$

After discretization, the collection of all possible solutions forms the *solution manifold*. This manifold is parametrized by $\mathcal{P}$, so its intrinsic dimension is at most $\dim \mathcal{P}$. However, it is embedded in a discrete space whose dimension is N_h , which may contain millions of degrees of freedom. The classical reduced order approach approximates the solution manifold using a low-dimensional linear subspace and then solves the reduced problem using a Galerkin projection. However, this approach has a limitation. A low-dimensional linear subspace gives a good approximation only when the Kolmogorov n -width decreases sufficiently fast.

Why a distribution. As discussed in Section 1, most of these methods are deterministic.

When they are trained using mean squared error, the optimal prediction is the conditional mean

$$\mathbb{E}[\mathbf{u} \mid \boldsymbol{\mu}].$$

If the solution map $\mathcal{S}$ is a well-defined deterministic function, then this is exactly the quantity we want. However, if the input information does not determine one unique solution, then the conditional distribution contains several possible solutions. In this case, the conditional mean can average between different possibilities and may not represent any of them well. For this reason, instead of predicting only one solution, we model the conditional distribution

$$p(\mathbf{u} \mid \boldsymbol{\mu})$$

and generate samples from it. The conditioning variable is $\mathbf{c} = \boldsymbol{\mu}$, while the target sample is $\mathbf{x}_1 = \mathbf{u}_h$. Therefore, the learned velocity field $\mathbf{v}_\theta(\mathbf{x}, t; \boldsymbol{\mu})$ defines a conditional sampler for the whole family

$$\{p(\cdot \mid \boldsymbol{\mu})\}_{\boldsymbol{\mu}}.$$

This also gives an offline/online structure similar to classical reduced order modelling. The expensive training stage is performed once. After that, samples for a new parameter $\boldsymbol{\mu}$ can be generated by solving the learned ODE.

2.2.1 Forward problems

The forward problem corresponds to solving (15) from the given parameters to the solution. Three of the benchmarks used later are forward problems.

For Poisson, we consider the source-to-solution map $f \mapsto u$. For Burgers, we consider the map from the initial condition to the final solution. For Darcy, we consider $(\kappa, f) \mapsto u$. These forward problems are shown in the corresponding rows of Figure 2. This setting is important because it is exactly the type of operator-learning problem where deterministic methods already perform very well. For example, the Fourier neural operator reaches approximately one percent relative L^2 error on the standard Darcy and Burgers benchmarks [19]. Therefore, a generative model that only reaches similar pointwise accuracy does not automatically provide an advantage. Generative methods can still be useful for forward problems, but not necessarily because of uncertainty.

Remark 5 (The forward posterior is a Dirac). *For a well-posed forward problem, the conditional distribution $p(\mathbf{u} \mid \boldsymbol{\mu})$ is a point mass located at $\mathcal{S}(\boldsymbol{\mu})$. Therefore, any non-zero spread produced by a conditional generative model is not physical uncertainty. It is model error. For this reason, the forward experiments in this report are not used to make uncertainty quantification claims. All uncertainty-related conclusions are based on the inverse problems discussed in Section 2.2.2. The forward experiments are instead used to check whether the solution operator is learned correctly.*

Even though the forward problem does not contain posterior uncertainty, it is still useful for evaluating the generative model. The main advantage is that the exact target solution is available for every input. Therefore, we can directly measure the error of the learned transport. It is also useful for studying sampling cost. Since sampling requires solving the learned ODE, the computational cost depends on the number of function evaluations. Therefore, the forward setting allows us to study the relation between sampling accuracy and computational cost using the same trained model. This is also the appropriate setting for evaluating properties such as straightness, reflow and the choice of ODE solver.

2.2.2 Inverse problems

The inverse problem reverses (15). Instead of computing the solution from the parameters, we try to recover the parameters from observations of the solution. We write the observation model as

$$\mathbf{y} = \mathcal{O}(\mathcal{S}(\boldsymbol{\mu})) + \boldsymbol{\eta}, \quad \boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I). \quad (16)$$

Here, $\mathcal{O}$ is the observation operator, which may only observe part of the solution.

A distribution is needed because inverse problems are generally ill-posed. Information can be lost through partial observations, measurement noise, or because different parameters may produce similar solutions. Therefore, one observation $\mathbf{y}$ may correspond to many possible values of $\boldsymbol{\mu}$. In this case, predicting only one value or only the posterior mean loses important information.

Darcy flow gives a simple example. The coefficient κ enters the equation

$$-\nabla \cdot (\kappa \nabla u) = f$$

through the flux. Where ∇u is small, the observations contain less information about κ . Therefore, the uncertainty of the recovered coefficient can change across the domain. Posterior samples can represent this variation, while the conditional mean averages the different possibilities into one field. The classical Bayesian approach often uses Markov chain Monte Carlo. However, MCMC requires many forward solves for each observation, since every likelihood evaluation involves the PDE model. Moreover, a new chain is needed for every new observation. There are two common alternatives.

Route (a). Train an unconditional model for $p(\mathbf{u})$ and guide the sampling procedure using the observation $\mathbf{y}$ at inference time [4]. The advantage is that the same prior model can be used with different observations. However, every new observation still requires a guided sampling procedure and likelihood gradients.

Route (b). Train a conditional model directly on simulated pairs $(\mathbf{y}, \boldsymbol{\mu})$ and learn

$$p(\boldsymbol{\mu} | \mathbf{y}).$$

This is a simulation-based inference approach. It requires samples from the forward model during training, but no explicit likelihood evaluation is needed. Wildberger et al. [31] show that conditional flow matching can be used for this purpose. FlowPDE follows route (b). The main reason is amortization. The expensive forward simulations and training are performed once. After training, a new observation can be mapped to posterior samples using only the learned ODE. This gives the same offline/online structure used in reduced order modelling. The limitation is that the learned model depends on the observation operator and noise level used during training.

Checking that the posterior is right. If a model predicts a distribution, it should also be evaluated as a distribution. Relative L^2 error of the ensemble mean only measures pointwise accuracy. It does not tell us whether the predicted uncertainty is calibrated. Therefore, we use two types of diagnostics.

Proper scoring rules. Proper scoring rules measure accuracy and uncertainty together [10]. The continuous ranked probability score (CRPS) is used for scalar predictions, while the energy score [29] extends this idea to multivariate outputs. This is important for PDEs because pointwise scores do not measure spatial correlations. The energy score can penalize samples that have correct marginals but unrealistic spatial structure.

Calibration diagnostics. Calibration measures whether the predicted spread agrees with the actual error. Coverage compares nominal and empirical coverage. The rank histogram [12] measures the position of the true solution inside the ensemble. A flat histogram indicates good calibration, while U-shaped and dome-shaped histograms indicate under-dispersion and over-dispersion. The spread-skill ratio [8] compares ensemble spread with the error of the ensemble mean.

3 FlowPDE

Section 2 described a design space. This section describes the software we wrote to explore it.

`flowpde` is a PyTorch library for training and sampling conditional continuous-time flows on PDE data. Its pieces are *separate*. The dynamics, the objective that fits them, the network that parametrizes them, the data, and the integrator that samples them are five different concerns, and each one can be replaced without touching the others. Listings below are quoted from the source, shortened, with the cuts marked.

3.1 Library overview

The design goal is best stated by example. Listing 1 is a complete training and sampling run on the 2D Darcy forward problem. There is no configuration file, no framework, and no per-problem training script.

```
from flowpde import NeuralODEFlow, FlowMatchingObjective, UNet, Trainer
from flowpde.datasets.exponax import DarcyGenerator
from flowpde.datasets import FieldNormalizer
from flowpde.trainers import FlowEvaluator

# 1. Data: (kappa, f) -> u on a 64x64 grid.
gen = DarcyGenerator(num_points=64, kappa_alpha=2.0, kappa_tau=3.0)
train_ds = gen.generate(num_samples=1000, seed=0, problem='forward')
val_ds = gen.generate(num_samples=200, seed=1, problem='forward')

# 2. Normalization: fit on train, share the SAME instance with val.
normalizer = FieldNormalizer.from_dataset(train_ds)
train_ds.set_normalizer(normalizer); val_ds.set_normalizer(normalizer)
```

```

# 3. Velocity field, dynamics, objective.
model = UNet(spatial_dim=2, spatial_size=64,
             solution_channels=1, condition_channels=2, base_channels=64)
flow = NeuralODEFlow(model, target_key='target', condition_key='input')
objective = FlowMatchingObjective(flow, path='linear', time_sampler='uniform')

# 4. Train, selecting on sampled error rather than on the loss.
evaluator = FlowEvaluator(objective, val_loader, normalizer=normalizer,
                          target_fields=val_ds.target_fields)
trainer = Trainer(objective, torch.optim.Adam(model.parameters()), lr=1e-3,
                  ema_decay=0.999, validator=evaluator, monitor='rel_l2',
                  checkpoint_extra={'normalizer_state': normalizer.state_dict()})
trainer.train(train_loader, epochs=200, save_dir='runs/darcy')

# 5. Sample: n_steps is the accuracy/cost dial of Section 2.1.3.
u = objective.sample(condition=batch['input'], n_steps=50, solver='euler')

```

Listing 1: A complete run. Every line is a modelling choice

The central separation. The important structural point is in step 3. `NeuralODEFlow` and `FlowMatchingObjective` are two objects and not one. This is Remark 1 made concrete.

The flow owns the *dynamics*, namely the velocity field, the integration, and the exact log-likelihood by (4). The objective owns the *training procedure*. So when we swap `FlowMatchingObjective(flow)` for `MaximumLikelihoodObjective(flow)`, we change how the same field is fitted and nothing else: same model, same data, same normalization, same sampler.

This means the comparison between simulation-free and simulation-based training is a controlled experiment rather than a comparison between two codebases, and that is what Section 4 uses. The reverse rule is enforced too: no training logic in the flow, and no dynamics in an objective.

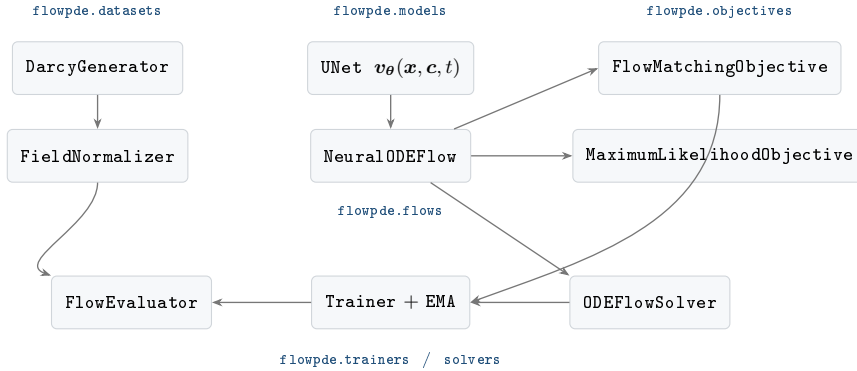


Fig. 1. Module dependencies. Arrows point from a component to the one that uses it. The flow depends on a network and nothing else, the objectives depend on the flow, and the trainer depends only on an object exposing `compute_loss` and `model`, so it does not care which objective it is given.

3.2 Modules and implementation details

We now go through the modules one by one, in the order in which they depend on each other.

3.2.1 flowpde.core

This module holds three abstract bases, `BaseFlow`, `BaseSolver` and `BaseConditioner`.

The most important thing `BaseFlow` provides is a tensor contract rather than an abstract method. `_extract_target_condition` flattens target and condition to (B, d) before anything else sees them, so a field on a 64×64 grid reaches the objective as a vector of length 4096, and the convolutional backbones reshape it back internally. The benefit is that flow, objective, path, coupling and solver are all written once in $\mathbb{R}^d$, with no special handling of spatial structure. The cost is that shapes must be declared to the backbone at construction, and Remark 6 returns to that.

`BaseConditioner` abstracts *how* the condition enters the network. `ConcatConditioner` concatenates c to x along channels, and `NullConditioner` discards the condition entirely.

3.2.2 flowpde.models

There are four backbones, and all of them use the same interface, `forward(x, f, t)`. Since the flow works with flattened vectors, each model reshapes the input internally when needed.

MPL is fully connected and is mainly used when spatial structure is not important. `ConvNet` uses time-conditioned residual convolutional blocks at fixed resolution. `ResNet` [13] uses a deeper residual architecture, while `UNet` [27] uses encoder-decoder skip connections and optional attention at the bottleneck.

The depth of the `UNet` is chosen according to the spatial resolution as $\min(5, \lfloor \log_2 S \rfloor - 2)$ so that downsampling stops at a spatial size of around four to eight pixels instead of going down to one. One common feature of all four models is the time conditioning. Time is encoded using a sinusoidal embedding and added residually inside each block. Therefore, every level of the network receives information about the current time.

3.2.3 flowpde.flows

`NeuralODEFlow` implements the continuous normalizing flow formulation of Section 2.1.1. It solves (1) using `torchdiffeq` and computes the conditional log-likelihood by augmenting the state with $\log p$. The trace term is computed either exactly using d Jacobian-vector products or approximately using the Hutchinson estimator (6).

Remark 6 (A known rough edge). *Since the flow uses flattened vectors, it must know the target dimension when it is different from the condition dimension. This happens in Darcy, where the condition contains two channels, $[\kappa, f]$, while the target u has only one. Currently, `flow._target_dim` is set during `compute_loss`. Therefore, if a checkpoint is loaded directly for inference, this value may not have been set yet. In that case, `sample()` falls back to the condition dimension. The current workaround is to pass `target_shape=` explicitly. Maybe a better solution would be to define the target shape when the flow is constructed.*

3.2.4 flowpde.objectives

`FlowMatchingObjective` composes the four components of Section 2.1.2. Each is accepted either as a registry name or as a constructed instance, which makes the API config-friendly without making configuration a dependency. Its entire training step is Listing 2, and it corresponds to (10) line by line.

```
def compute_loss(self, batch, target_key=None, condition_key=None, **kwargs):
    x_1, condition = self.flow._extract_target_condition(batch, ...)

    # Passing the batch lets BatchSource return the precomputed x_0
    # that reflow depends on, instead of drawing fresh noise.
    x_0 = self.sample_base_distribution(x_1.shape, self.model_device, batch)

    # A source carrying its own pairing already fixes which x_0 goes with
    # which x_1, so re-coupling here would destroy it.
    if not self._source_defines_pairing(batch):
        x_0, x_1 = self.coupling(x_0, x_1)

    t = self.time_sampler(batch_size, self.model_device)
    x_t, v_target = self.path(x_0, x_1, t) # interpolate + velocity
    v_pred = self.model(x_t, condition, t)
    return F.mse_loss(v_pred, v_target)
```

Listing 2: `FlowMatchingObjective.compute_loss`: (10) in eight lines.

Table 1 lists the components that can be replaced at each part of the implementation. The two guarded branches enforce the conditions discussed in Remarks 2 and 4. The path returns the interpolated point and its velocity together. Therefore, the interpolation and the target velocity cannot become inconsistent. Similarly, the coupling step is skipped when the source already defines the pairing. This prevents minibatch optimal transport from changing the source-target pairs used by reflow. Finally, `BatchSource` uses `strict=True` by default. If the required key is missing, it raises an error instead of silently resampling.

Slot	Options	What it controls
Path	<code>linear</code> , <code>ot_conditional</code>	the interpolant and its velocity
Time	<code>uniform</code> , <code>logit_normal</code> , <code>beta</code>	where capacity is spent in t
Coupling	<code>independent</code> , <code>minibatch_ot</code>	which x_0 meets which x_1
Source	<code>gaussian</code> , <code>batch</code>	where trajectories start

Table 1. The four flow-matching slots. Each accepts a registry name or an instance, so the variants of Section 2.1.2, namely standard flow matching, rectified flow, OT-CFM and minibatch-OT coupling, are configurations rather than subclasses.

The second objective, `MaximumLikelihoodObjective`, fits the same flow by maximizing (5) instead. It shares the model, the data and the sampler, which is what makes the comparison in Section 4.2 a controlled one.

3.2.5 `flowpde.trainers`

`Trainer` can work with any object that provides `compute_loss(batch)` and a `.model`. Therefore, it is not tied to one specific training objective. Two parts of the trainer are particularly important.

Exponential moving averaging. The flow-matching gradient is naturally noisy. As discussed in Theorem 1, the target velocity is not uniquely determined by the network input, so the optimization does not settle around one exact target. For this reason, we use exponential moving averaging of the model weights. The averaged weights are used during evaluation and are also the final weights that we save. The averaging rate is increased gradually using $\min\left(d, \frac{1+n}{10+n}\right)$, which avoids introducing a separate threshold for when averaging should start.

Selecting on sampled error instead of the training loss. This is one of the most important choices in the trainer. As explained in Section 2.1.2, the flow-matching loss has an irreducible minimum. Therefore, two models can have very similar training losses while producing samples with quite different accuracy. For this reason, `FlowEvaluator` evaluates the actual generated samples. It integrates the ODE, denormalizes the output, and compares it with the ground truth. `Trainer` then selects the best checkpoint using this sampled error. The evaluation noise is generated using a fixed seed. Therefore, changes in the evaluation score between epochs mainly come from the model instead of a different random sample.

When `ensemble_size > 1`, the evaluator also computes the ensemble mean and spread. The uncertainty metrics discussed in Section 2.2.2 are implemented in `flowpde.utils.uq_metrics`. These include coverage, reliability curves, rank histograms, spread-skill with the $\sqrt{(K+1)/K}$ correction, CRPS and the energy score.

Reflow and the pairing condition. As discussed in Remark 4, reflow requires the same source sample $\mathbf{x}_0$ to remain paired with the target that it generated. For this reason, `BatchSource` uses `strict=True`. If the required source sample is missing, the code raises an error instead of sampling a new one. We tested this behavior on the Burgers ConvNet using three random seeds and two reflow iterations. As a control, we also tested the incorrect version where $\mathbf{x}_0$ is sampled independently again. This is exactly the pairing error that the implementation is designed to prevent.

3.2.6 `flowpde.datasets`

Data generation is implemented using Exponax [16] on top of JAX [2]. `PoissonGenerator` solves $\nabla^2 u = f$ using a spectral method. `BurgersGenerator` evolves an initial condition using an exponential time-differencing scheme. The viscosity can be sampled independently for each example as $\nu \sim \text{LogUniform}$ although in the experiments of Section 4 we fix $\nu = 10^{-2}$. `DarcyGenerator` solves $-\nabla \cdot (\kappa \nabla u) = f$ using second-order finite differences. The linear system is solved with a fixed-step conjugate gradient method written with `jax.lax.scan`. This allows the full generator to be vectorized with `jax.vmap` without using a Python loop over samples. The coefficient κ is generated as a log-normal Gaussian random field,

$$\kappa = \exp(cg), \quad g \sim \mathcal{N}(0, (\tau^2 I - \Delta)^{-\alpha}). \quad (17)$$

This guarantees that $\kappa > 0$ everywhere. With $\alpha = 2$ and $\tau = 3$, the construction follows the Darcy setup used in the FNO benchmark [19].

Every generator accepts either `problem='forward'` or `problem='inverse'`. The forward setting uses the physical parameter or forcing as input and the PDE solution as target. The inverse setting swaps these roles. For inverse problems, additive observation noise can be controlled through `obs_noise_std`, while random spatial masking is controlled through `obs_mask_fraction`. One detail is important here. We keep the observation mask as part of the input. Otherwise, a zero value would be ambiguous: it could either mean that the field is actually zero or that the point was not observed. For this reason, the mask is appended as an additional conditioning channel. The model therefore has to use `condition_channels` consistent with `sample['input'].shape[0]`.

Normalization. Flow matching transports samples from $\mathcal{N}(\mathbf{0}, I)$ to the target distribution. Therefore, large differences in the scale of the target fields can make the regression problem in (10) harder to optimize. `FieldNormalizer` standardizes each field separately. Two implementation choices are important. First, the normalization statistics are stored using the original field names, such as `'source'`, `'solution'` and `'kappa'`. They are not stored according to whether the field is currently used as input or target. This means

that the same normalizer can be used for both 'forward' and 'inverse' problems, even though the roles of the fields are swapped. Second, fields without registered statistics are left unchanged. This is important for `obs_mask`, since the mask should remain binary instead of being standardized like a continuous field. All reported metrics are therefore computed in physical units. Before evaluation, predictions are mapped back using `denormalize_channels`. Also, if a normalizer is provided but `target_fields` is missing, `FlowEvaluator` raises an error instead of evaluating the prediction in standardized units.

Remark 7 (Never refit on validation). *The normalizer is fitted once using `FieldNormalizer.from_dataset(train_ds)` and the same normalizer instance is then used for the validation and test sets. Fitting a new normalizer on validation or test data would use information from the held-out set and therefore introduce data leakage.*

3.2.7 flowpde.solvers

`ODEFlowSolver` wraps `torchdiffeq` and provides both fixed-step and adaptive ODE solvers. The fixed-step methods include `euler`, `midpoint` and `rk4`, while adaptive methods include `dopri5` [6], `tsit5` and others. During integration, the condition is kept fixed while only the state $\mathbf{x}$ changes. Therefore, the solver follows the formulation of (1), where the dynamics act on the state and not on the conditioning variable. The solver and the number of steps are chosen at sampling time, after training. This means that we can evaluate different accuracy–cost choices using the same trained model. This is what allows the comparison in Section 4.3 without retraining the network. For this reason, the ODE solver is implemented as a separate module rather than being fixed inside the flow.

4 Numerical experiments

In this section, we test whether the conditional flow can learn the parametrized PDE solution map discussed in Section 2.2. We first check whether the forward map is learned correctly. Then, we study the effect of the ODE solver and the number of sampling steps. After that, we move to inverse problems, control the level of ill-posedness, and finally compare the learned sampler with MCMC. Throughout the experiments, we try to change only one component at a time while keeping the remaining setup fixed. The datasets, train–validation–test splits, normalization, optimizer, training schedule, EMA, sampler and evaluation seed are kept the same whenever possible. Therefore, differences between experiments can mainly be attributed to the component being changed. When this is not possible, we state it explicitly.

4.1 Setup

Benchmarks. We use three PDE families: Poisson, Burgers and Darcy. For Poisson, $\nabla^2 u = f$ the forward problem maps a random source field f to the solution u . The problem is solved on a 64×64 grid using two different spectral widths, which are discussed later.

For Burgers, the forward problem maps the initial condition to the solution at $T = 0.4$ with fixed viscosity $\nu = 10^{-2}$. Both Poisson and Burgers are also considered in the inverse direction in Section 4.5. For the inverse Poisson experiments, we use a 32×32 grid. Darcy is the main benchmark used in most of the experiments. We solve $-\nabla \cdot (\kappa \nabla u) = f$ on $[0, 1]^2$ using a 64×64 grid. The coefficient κ is generated from the log-normal random field defined in (17), with $\alpha = 2, \tau = 3$, which follows the standard FNO Darcy benchmark [19]. The source f is sampled as a random Fourier field with cutoff 8, and the linear system is solved using 500 conjugate-gradient iterations. This gives the six benchmark settings shown in Figure 2.

What is held fixed. Every run trains a conditional `NeuralODEFlow` on the standard flow-matching configuration, meaning a linear path, a uniform time sampler, an independent coupling and a Gaussian source, with AdamW at 2×10^{-4} , cosine decay, gradient clipping at 1.0, and EMA at $\rho = 0.999$. Splits are 3000/200/200 at fixed generator seeds, so every variant sees identical data. One `FieldNormalizer` is fitted on train and shared (Remark 7). Selection is on sampled validation error at a fixed seed and never on the training loss. Table 2 records what each experiment varies.

Metrics. Errors are relative L^2 in physical units on the held-out split, with relative H^1 , MAE and a normalized maximum error recorded alongside. Distributions are scored with CRPS, the energy score, coverage and the bias-corrected spread–skill ratio over $K = 32$ draws at a fixed ensemble seed.

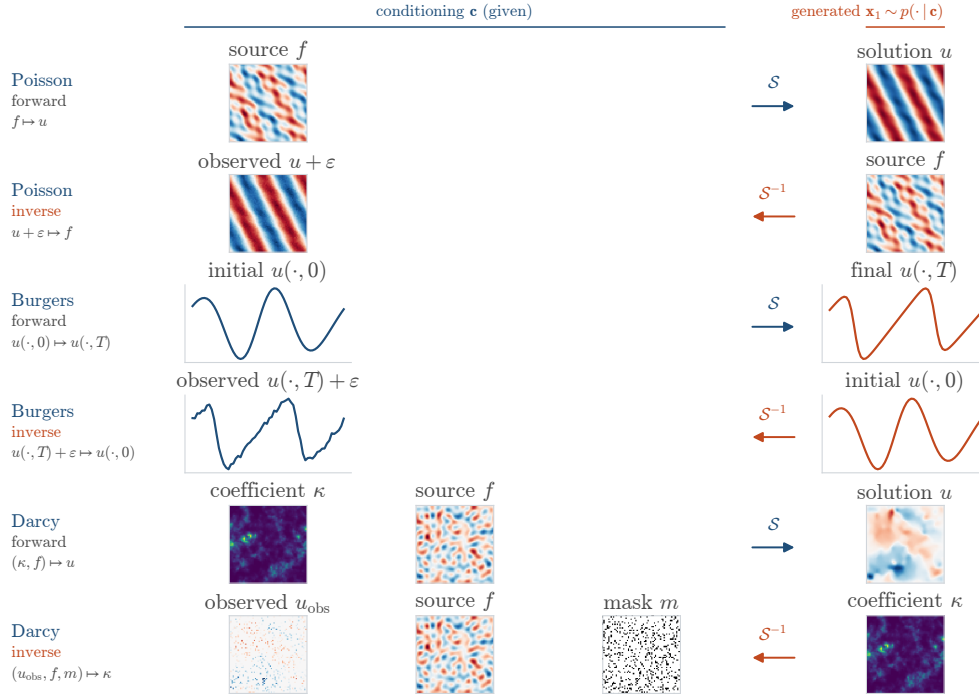


Fig. 2. The six problem settings. Each row is one conditional distribution the library trains: everything left of the arrow is conditioning, stacked as channels of a single input tensor, and the field on the right is what the flow generates. Forward rows apply the solution operator $\mathcal{S}$, inverse rows invert it. The inverse rows show the observation the model is actually given: Poisson and Burgers under additive noise, Darcy at 15% observed with its mask carried as a third conditioning channel. Poisson is drawn in the widened source regime. Fields are from the seeded test generators, so they are the distributions the models below were scored on.

	Problem	Varied	Levels	Epochs
<i>Section 4.2: conditioning, backbone, objective</i>				
E1	Darcy 2D, fwd.	conditioning	null, concat	500
E2	Burgers 1D, fwd. [†]	backbone	ConvNet, ResNet, UNet, UNet ⁻	500
E3	Darcy 2D, fwd.	backbone	ConvNet, ResNet, UNet, UNet ⁻	500
E8	Poisson 2D, fwd.	backbone, source spectrum	2×2	150
E4	Darcy 2D, fwd. $16^{2\dagger}$	objective	flow matching, max. likelihood	50
<i>Section 4.3: sampling cost</i>				
E7	Burgers 1D, fwd. [†]	path, schedule, coupling	6 configurations	500
<i>Sections 4.4 and 4.5: posteriors</i>				
E5	Darcy 2D, inverse	conditioning, target	null/concat $\times \kappa/(\kappa, f)$	100, 500
E6	Darcy 2D, inverse	backbone	ConvNet, ResNet, UNet, UNet ⁻	500
E9	Darcy 2D, inverse	observed fraction, noise	$4 + 4$ levels	100
E11	Poisson 2D, inv. 32^2	observation noise	4 levels	500
E12	Burgers 1D, inverse	observation noise	4 levels	500
<i>Section 4.6: comparison against MCMC</i>				
E10	Darcy 2D, inverse	sampler: pCN reference	4 obs. $\times$ 2 chains	—
E13	Burgers 1D, inv. 256	sampler, identifiability	2 regimes $\times$ 4 obs.	150, 500
<i>Re-evaluations on finished weights, no retraining</i>				
R1	Burgers, Darcy, Poisson	solver, step count	4 solvers, 8 step counts	—
R2	Burgers, Darcy	straightness mode	trajectory, interpolant	—
R3	Darcy, both directions	ensemble size	$K = 32$	—

Table 2. The experiment matrix. UNet⁻ is the UNet without attention. [†] marks the experiments repeated at three matched seeds (42, 1042, 2042) per cell, and the rest are single runs. R1–R3 are measurements on finished weights, which need no retraining because sampling cost, straightness and ensemble size are all chosen at inference time.

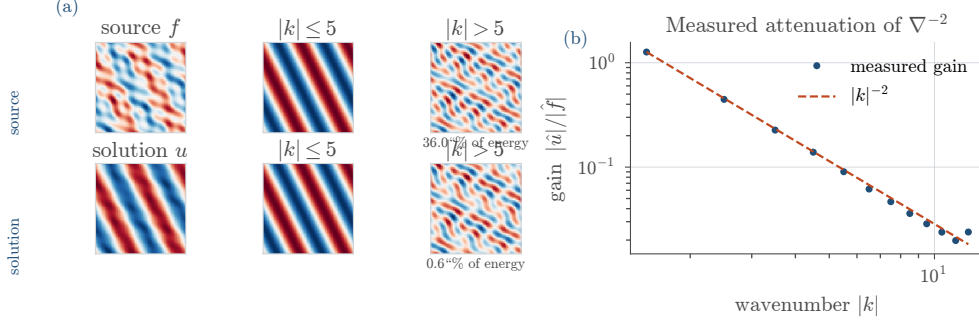


Fig. 3. Why the inverse direction is the harder one. The forward Poisson operator has spectral gain $|k|^{-2}$, so inverting it multiplies the high-wavenumber band by $|k|^2$, and that band is where observation noise lives. The parametrization is recoverable only to the extent that the observation constrains it, which is what Section 4.5 turns into a controlled variable.

Hardware. Training on one CUDA device for E1–E10, and on Apple silicon for E11–E13. Re-evaluations (R1–R3), solver timings and the MCMC chains ran on CPU. Wall-clock numbers come from more than one machine, so they are comparable only within a table.

4.2 Conditioning, backbone and objective

Before anything else we check two things: that the conditioning tensor, which holds the PDE parameters, really reaches the velocity field, and that the resulting map is accurate. Both are for verification, so we report them briefly.

The conditioning is actually used. If the condition never influences the velocity, the model still learns a good *unconditional* generator of Darcy solutions, its loss still decreases, and its samples still look like plausible fields. E1 replaces `ConcatConditioner` with `NullConditioner` and changes nothing else.

On the same 200 test samples the conditioned model is better on *every one*, with a median per-sample ratio of 17.2, and median relative L^2 falls from 1.30 to 0.092. Both loss curves converge and both look healthy.

Two things follow. First, this is the concrete reason we select on `FlowEvaluator` rather than on the loss. Second, it gives us a scale for the rest of the section, since the null model is an honest unconditional sampler, so 1.30 is what learning nothing looks like here.

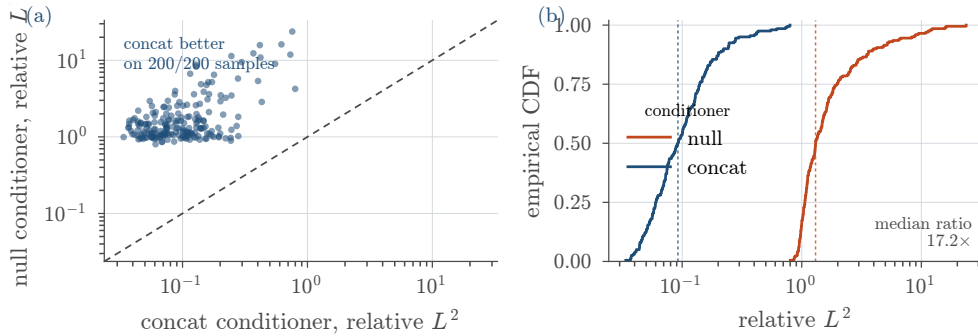


Fig. 4. Conditioning on Darcy 2D, 200 paired test samples. (a) Per-sample relative L^2 of the null conditioner against the concat conditioner; every point lies above the diagonal. (b) The same data as empirical CDFs, with dashed lines at the medians. Both models minimize the same objective and both loss curves converge, so the difference is invisible without sampling.

The smallest backbone is the most accurate. E2, E3 and E8 only change the network used for v_θ . On Burgers, ConvNet has 0.31M parameters and reaches relative L^2 error 0.021, while UNet has 2.34M parameters and reaches 0.266. Therefore, ConvNet is about 12.4 $\times$ more accurate. The same trend appears on Darcy. ConvNet reaches 0.073, while UNet reaches 0.128, and ConvNet is better on 176 of the 200 test samples. Attention also does not help much. UNet and UNet⁻ differ by less than 1.5% on Darcy and give almost the same result on Burgers. To check the effect of random seeds, we repeated E2 with three matched seeds. The results are ConvNet 0.0214 ± 0.0003 , ResNet 0.1420 ± 0.0026 , UNet 0.2657 ± 0.0017

and UNet -0.2660 ± 0.0013 . The seed variation is much smaller than the difference between the models. Therefore, the model ordering is stable, and attention does not give a clear improvement.

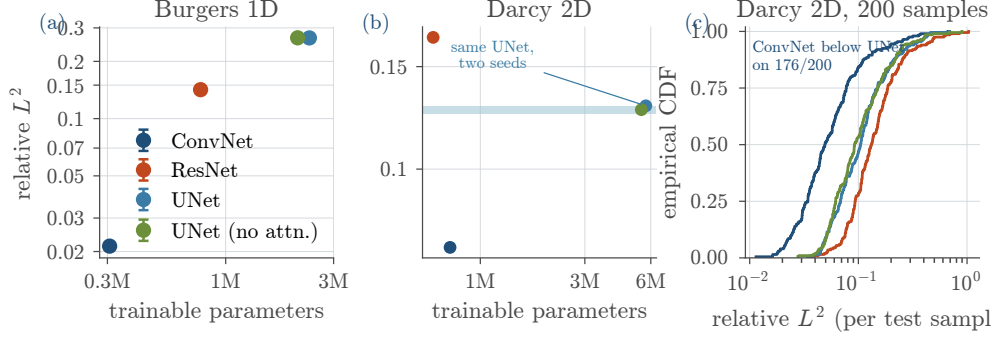


Fig. 5. (a, b) Accuracy against parameters; the ordering is inverted with respect to capacity. Error bars in (a) are the standard deviation over three matched seeds, and they are smaller than the markers. The band in (b) spans two runs of the *identical* UNet at different seeds, which is the only replicate available on Darcy. (c) Per-sample errors on Darcy.

There is one place where the ordering reverses. On E8’s widened Poisson source the UNet wins by 15%, because its multi-scale structure helps once the source spans more modes. One other thing should be recorded. On Burgers, above $|k| = 13$, every model is wrong by a factor between 48 and 400, in a band that carries too little energy to show up in relative L^2 . That is the observation behind PDE-Refiner [22], and here we measure it rather than assume it.

Flow matching against maximum likelihood. E4 fits one flow, one UNet of 40,777 parameters, one dataset and one sampler in two ways, changing only `FlowMatchingObjective` for `MaximumLikelihoodObjective`. The problem is deliberately small, Darcy at 16×16 , because the simulation-based route is not affordable at 64×64 .

The result is that each objective wins on its own metric. Maximum likelihood reaches -1.86 nats per dimension against -1.21 . Flow matching reaches relative L^2 0.610 against 1.064, and it does so at 1.92 seconds per epoch against 135.2, a factor of 70, using 18.6MB of peak memory against 166.3.

So the conclusion is narrower than “flow matching is better”. At a fixed small budget the simulation-free objective gives better samples and more reproducible runs, the simulation-based one gives better density estimates, and the difference in compute between them is two orders of magnitude. Everything below uses flow matching for that reason.

4.3 Accuracy against sampling cost

Sampling is just an ODE solve, so we choose how many evaluations to spend per solution *after* training, on frozen weights. Training is paid once, while sampling is paid every time we generate a solution. R1 sweeps this sampling budget, R2 measures the geometry of the paths behind it, and E7 shows that this geometry is something we configure as well.

We plot the error against *function evaluations* and not against steps. One RK4 step costs four evaluations, so a per-step comparison would make RK4 look four times better than it is.

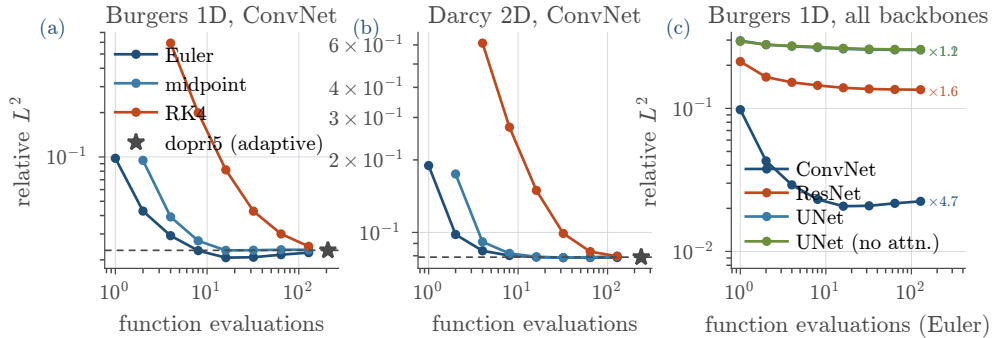


Fig. 6. Accuracy against cost on frozen weights. (a, b) The best backbone per problem; the star and dashed line mark the converged solution of the learned ODE. (c) Euler on every Burgers backbone. Steep means discretization error dominates, flat means the field itself is the limitation.

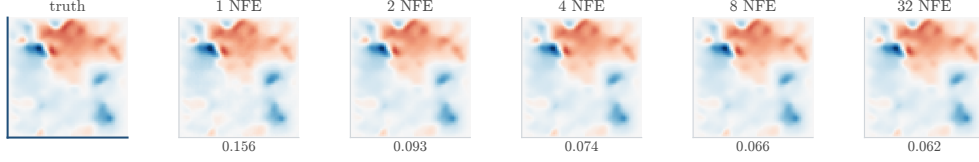


Fig. 7. One Darcy test condition, one fixed $\mathbf{x}_0$, integrated with Euler at increasing budgets; the number below each panel is that sample’s relative L^2 . A single evaluation already recovers the structure of the solution, which is what makes the left end of Figure 6 usable.

At the same budget, Euler is better than the higher order solvers. On Burgers with the ConvNet, Euler and RK4 differ by $8.6\times$ at 8 evaluations (0.0231 against 0.1992), by $3.9\times$ at 16 and by $2.1\times$ at 32. On Darcy the ratios are 3.4, 1.9 and 1.3, and below 64 evaluations every backbone on both problems has RK4 at or behind Euler.

Solving the ODE exactly is not what we want either. The ConvNet on Burgers is best at 0.0207 with sixteen Euler evaluations. The same weights integrated to convergence (`dopri5`, 208 evaluations) give 0.0232, which is 11% *worse*. Coarse Euler ends up closer to the data than the exact solution of the learned ODE, because its truncation error partly cancels the error of the model. This happens elsewhere too, but not always: on Darcy the ResNet gains 6% this way, while the two UNets match their converged error to four decimals. So we only claim the negative part, which is enough for us: the converged solution of the learned ODE is nothing special, and reporting accuracy “at 50 steps” gives one point of a curve that is neither monotone nor the same from model to model.

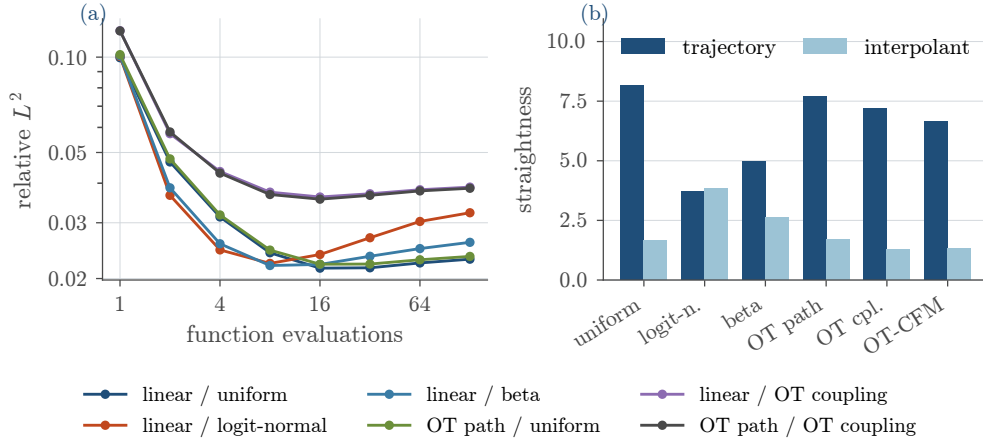


Fig. 8. (a) Error against function evaluations, Euler, averaged over three seeds. The curves cross: the configuration that wins below eight evaluations loses above sixteen. (b) The two straightness definitions disagree about which configuration is straight, and only the trajectory one predicts (a).

Two negative results belong here as well.

The first one is that optimal transport coupling did not pay. Minibatch-OT coupling and OT-CFM are worse than the baseline at *every* budget, 0.0375 and 0.0372 converged against 0.0214, which is the opposite of what we expected. Figure 8(b) shows why. OT coupling gives by far the straightest *interpolant*, 1.28 against 1.66, but the trajectories of the model stay as curved as the baseline ones. It straightened the training target and not the learned dynamics. This is why Definition 1 is stated on trajectories: with interpolant straightness alone we would have concluded that OT coupling worked.

The second one is that reflow straightened even when it was broken. The reflow experiment of Section 3.2 straightened under a deliberately broken pairing almost as much as under the correct one, $8.23 \rightarrow 5.81$ against $8.23 \rightarrow 5.18$, and improved few-step accuracy in neither case. So straightening by itself is not evidence that a method worked.

4.4 Inverse problems and calibration

E5 moves to the setting the generative argument was made for. The observation model is (16), where $\mathcal{O}$ is a random restriction that keeps 15% of the grid and $\sigma = 2 \times 10^{-4}$. The target is either κ alone, conditioning on (u_{obs}, f, m) , or the pair (κ, f) jointly.

First we need a reference level. A null-conditioned model is here a generative *prior* over κ : it produces admissible fields which have nothing to do with the observation, and it scores single-draw relative L^2 of

1.069 and 1.104. This is what “no information was used” looks like on this benchmark, so we keep one of these rows in every comparison below.

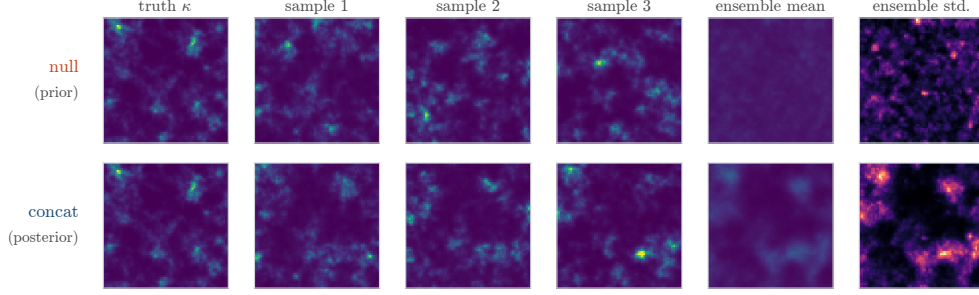


Fig. 9. Posterior draws for one test condition, joint (κ, f) target at 5000 samples, with the κ channel shown. The unconditional model produces admissible coefficient fields unrelated to the observation, and its ensemble mean is featureless. The conditioned model places structure where the truth has it, and its ensemble standard deviation is spatially organised rather than uniform, which is the quantity Figure 10 measures. The joint target is shown because the κ -only null run has no checkpoint.

A single draw hides half of what conditioning buys. A single draw from a correct posterior *should* differ from the truth by roughly the posterior width, so R3 draws $K = 32$ samples per condition and scores them properly, in Table 3. In both matched null-versus-concat pairs the proper scores show a larger effect than the point metric. On the κ target a single draw says conditioning buys 12.8% while CRPS says 18.6%; on the joint target the single draw says 13.8% and CRPS says 25.4%. So the conditioned models are not only closer to the truth, they also put a *better shaped* distribution on it, and if we look at relative L^2 alone, half of what conditioning bought stays invisible.

Target	Cond.	CRPS	ES	cov. 90	spr/skl	$\rho_{e,s}$	L^2 (1)
<i>Inverse, 5000 samples, 100 epochs</i>							
$\kappa^\ddagger$	null	0.860	95.9	0.826	0.90	−0.89	1.069
$\kappa^\ddagger$	concat	0.700	83.8	0.838	0.92	−0.36	0.932
(κ, f)	null	0.631	100.2	0.825	0.92	−0.05	1.104
(κ, f)	concat	0.471	84.8	0.847	0.94	+0.02	0.952
<i>Inverse, 5000 samples, 500 epochs</i>							
κ	concat	0.747	94.0	0.703	0.47	+0.38	0.794
<i>Forward, for contrast: true posterior is a Dirac</i>							
u (ConvNet)	concat	$3.5 \cdot 10^{-5}$	0.003	0.959	1.16	+0.88	0.075
u (UNet)	concat	$1.5 \cdot 10^{-4}$	0.012	0.651	0.22	+0.77	0.136

Table 3. Distributional scores, $K = 32$ at a fixed seed, in physical units. ES is the energy score. “spr/skl” is the spread–skill ratio after the $\sqrt{(K+1)/K}$ correction, where 1 means calibrated and below 1 means over-confident. $\rho_{e,s}$ is the rank correlation between predicted spread and actual error, and L^2 (1) is a *single* draw’s relative L^2 . Rows marked $\ddagger$ are the reference cell of E9, scored on all 200 test conditions at ensemble seed 123, and the rest are the R3 re-evaluation on 64 conditions at ensemble seed 7. Comparisons are made within a protocol.

Section 2.2.2 argued that κ enters only through the flux, so the observation constrains it where ∇u is large and barely at all where the solution is flat. Does the learned posterior know this? We bin every pixel of every test condition by local $|\nabla u|$ and check. The median posterior standard deviation falls from 0.907 in the lowest flux decile to 0.234 in the highest, a factor of 3.9, with rank correlation -0.53 over 260,000 pixels. The model was never told any of this.

This is our best result. The model does not simply return an error bar next to a point estimate. It returns a posterior whose spatial structure matches what the physics says about where κ can be recovered.

On the inverse problem the backbone stops mattering. E6 repeats the E3 backbone comparison in the inverse direction, and all four networks land between 0.788 and 0.820, so the worst one exceeds the best one by 4%, while on the forward problem the same four span a factor of 2.3. The reason is that once the observation determines the target only up to a wide posterior, the quality of the velocity field is not what limits the result anymore. The information in the data is. We reach the same conclusion from the other side in Section 4.3, where a flat error-against-budget curve tells us that the integrator was not the limit either.

Calibration gives us one result and one warning. If we run the same suite on two *forward* models, whose true posterior is a point mass and whose correct spread is zero, we get spread–skill ratios of 1.16

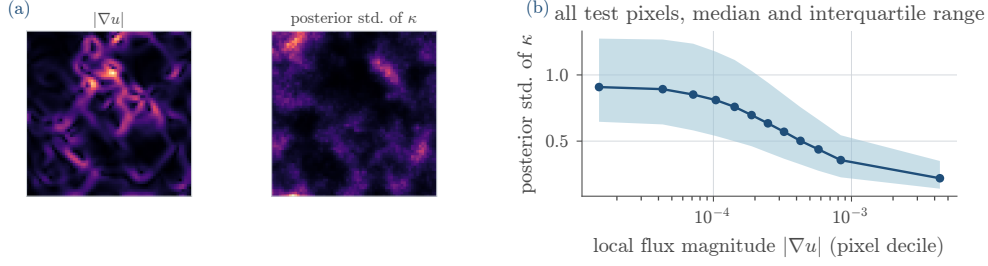


Fig. 10. Spatially varying identifiability. **(a)** $|\nabla u|$ and the posterior standard deviation of κ for one condition; the bright regions coincide. **(b)** All test pixels by flux decile: posterior width falls $3.9\times$ from lowest to highest.

and 0.22. Two models approximating the same posterior report uncertainties that differ by a factor of five, which makes Remark 5 quantitative. In practice it means we should always ensemble on a forward problem, since averaging 32 draws cuts the ConvNet’s relative L^2 from 0.075 to 0.035, and that we should never call the result uncertainty quantification.

The warning is the κ cell trained for 500 epochs instead of 100. Its single-draw error improves from 0.932 to 0.794, while its spread–skill ratio falls from 0.92 to 0.47 and its 90% coverage from 0.84 to 0.70, with the U-shaped rank histogram of under-dispersion in Figure 11(a). More optimization made the posterior narrower than the evidence supports. A point metric rewards this and a proper scoring rule penalizes it, and this is why we run both.

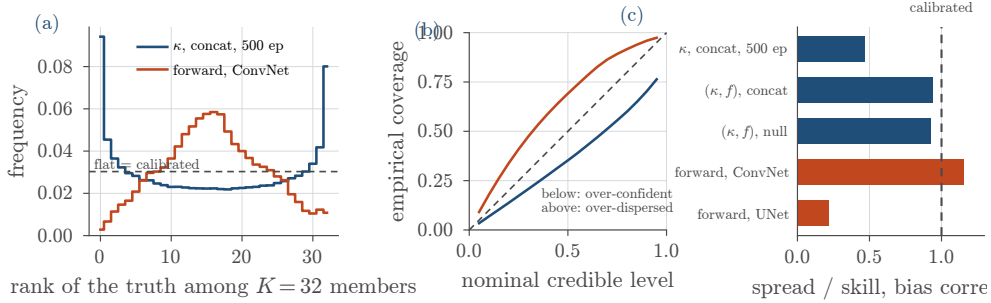


Fig. 11. Calibration, $K = 32$. **(a)** Rank histograms: the inverse model is U-shaped (under-dispersed) and the forward one dome-shaped. **(b)** Reliability against the diagonal. **(c)** Bias-corrected spread–skill; the two forward models, whose true posterior is identical and degenerate, disagree by a factor of five.

4.5 Controlling how ill-posed the problem is

Everything above observes 15% of the grid at one noise level, which makes the degree of posterior degeneracy an accident of a single setting. It does not have to be. The mask fraction and the observation noise are generator parameters, so the amount of information in the observation is something we can vary ourselves.

E9 varies it on Darcy, with eight cells trained identically and each of them scored with the full $K = 32$ suite. E11 and E12 then do the same on the two benchmarks that appeared until now only in the forward direction, which answers a question the Darcy sweep cannot: whether this behaviour belongs to the method or to that one problem.

Both ends of the sweep are anchored. Going from 5% to full observation moves CRPS from 0.772 to 0.437 and the ensemble-mean relative L^2 from 0.736 to 0.488, while going from noiseless to $\sigma = 2 \times 10^{-2}$ moves CRPS from 0.634 to 0.860. This last number is worth comparing with the unconditional prior, which also scores 0.860 to three decimals. So at a hundred times the reference noise the conditional model ends up *exactly* at the prior: it still produces admissible κ fields, but they carry no information about the observation. The upper end of the range is then not an arbitrary bad number. It is a measured floor, and it shows where a model lands once the data are destroyed.

Table 4 repeats the sweep on Poisson ($u \rightarrow f$) and Burgers ($u(T) \rightarrow u(0)$), fully observed so that noise is the only degradation, with the two ladders matched in units of the observation’s own standard deviation. Every column is monotone with no crossings. The clean Poisson inverse is nearly exact, at CRPS 0.0072 and ensemble-mean relative L^2 0.014, which tells us that the inverse direction is not hard in itself. It is hard exactly as much as the observation is uninformative. The two problems also degrade at very different rates, and Figure 3 predicts which one degrades faster: thirty per cent of relative noise multiplies the

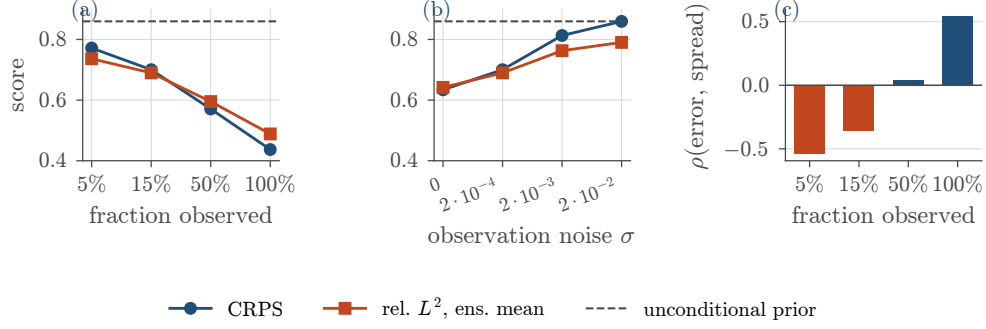


Fig. 12. Inverse Darcy, κ target, as the observation is degraded. **(a)** Observed fraction at $\sigma = 2 \times 10^{-4}$. **(b)** Noise at 15% observed. In both, the dashed line is the unconditional model, so a curve reaching it is a model that has stopped using its data. **(c)** Rank correlation between per-sample ensemble spread and per-sample error, which changes sign across the sweep.

CRPS of Poisson by 17.5 and the one of Burgers by 1.7, because inverting the Poisson operator amplifies exactly the band the noise occupies.

	σ	σ/std	rel. L^2		CRPS	cov. 90%	spread/skill
			1 draw	mean			
Poisson	0	0.00	0.047	0.014	0.0072	0.998	2.96
	0.0452	0.30	0.476	0.359	0.1267	0.815	0.91
	0.0905	0.60	0.775	0.572	0.2037	0.823	0.93
	0.1357	0.90	0.960	0.700	0.2519	0.830	0.95
Burgers	0	0.00	0.469	0.297	0.1257	0.879	0.89
	0.0769	0.30	0.810	0.553	0.2121	0.836	0.89
	0.1538	0.60	0.984	0.651	0.2451	0.828	0.90
	0.2307	0.90	1.102	0.723	0.2707	0.822	0.90

Table 4. Noisy inversion on Poisson ($u \rightarrow f$) and Burgers ($u(T) \rightarrow u(0)$). “1 draw” is a single posterior sample and “mean” the $K = 32$ ensemble mean, both in physical units on the test split. Coverage is against a nominal 0.90. One run per cell at seed 42.

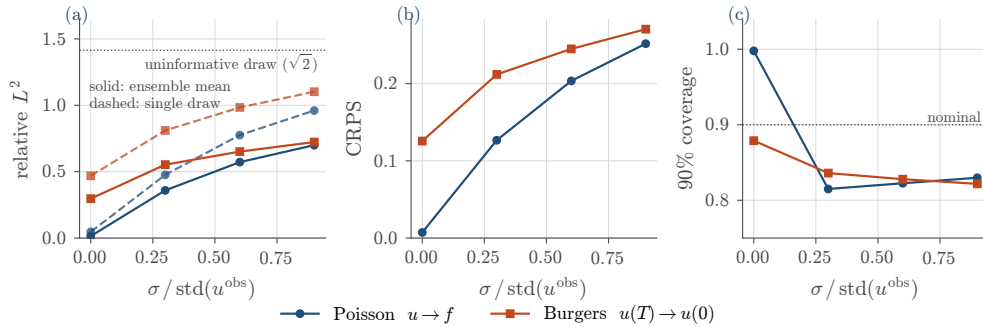


Fig. 13. Noise sweeps on both inverse problems, against noise measured in units of the observation’s own standard deviation so that the two benchmarks share an axis. **(a)** Relative L^2 : the ensemble mean (solid) beats a single draw (dashed) by 1.35–2.3 $\times$ everywhere. **(b)** CRPS. **(c)** 90% coverage: the clean Poisson cell is the outlier, and it is the one whose posterior is nearly a Dirac.

Calibration survives the degradation. Across all eight Darcy cells the corrected spread–skill ratio stays between 0.88 and 0.93 and the 90% coverage between 0.83 and 0.85, while CRPS nearly doubles, and the Poisson and Burgers ladders sit in the same band. So the models track how much they know even as what they know collapses. This is the property we wanted from the method, and it is worth keeping apart from accuracy: a model can be uninformative and still honest about it, and these ones are. The exception, in both directions, is a near-Dirac posterior. The clean Poisson cell at spread–skill 2.96 is the one *over*-dispersed model in the report, for the same reason the forward models of Section 4.4 are unreliable there.

One caveat applies to the whole subsection. Spread is a usable error bar only where the problem is well determined. The rank correlation between per-sample spread and per-sample error runs -0.54 at 5%

observed, -0.36 at 15%, $+0.04$ at 50% and $+0.54$ at full observation, so it changes sign. The reason is that where the observation is uninformative the model falls back towards the prior, and prior draws are both wide and, for a field near the prior mean, not especially wrong. Hence the widest predictions are not the worst ones. Sampler spread becomes a usable error bar only once the data determine the answer, which is unfortunately when we need it least.

4.6 Comparison against MCMC

Everything above shows that the inverse samplers are calibrated and that they use their data. Whether they approximate the *Bayesian* posterior is a different question, and to answer it we need a reference. Getting one is the expensive part.

The natural algorithm is preconditioned Crank–Nicolson [5], and it is natural for a reason specific to this generator. Here κ is built by pushing white noise $\xi \sim N(\mathbf{0}, \mathbf{I})$ through a fixed spectral filter (17), so if we sample in ξ instead of κ the prior is exactly Gaussian. Then it cancels from the acceptance ratio, and the proposal

$$\xi' = \sqrt{1 - \beta^2} \xi + \beta \zeta$$

preserves it exactly. Hence the acceptance rate does not degenerate under grid refinement, which is what makes a 4096-dimensional chain tractable at all.

On Darcy we could not obtain the reference. E10 ran four test observations, two chains each, 6000 burn-in and 20,000 sampling iterations, with one forward solve per chain per iteration. The sampler behaved as designed and acceptance settled at 0.20, but 52,000 forward solves per observation were not enough: split- $\hat{R}$ runs from 1.05 to 1.89 and the effective sample size from 20,000 draws is between 12 and 39. The clearest sign is that two chains targeting the *same* posterior disagree on the posterior mean by 0.38 to 0.99 relative L^2 , while two chains targeting *different* observations disagree by 1.04. So we draw no posterior comparison from this run, since a reference that disagrees with itself cannot be used as one. What the failure does give us is a cost: seven hundred and one seconds and 52,000 PDE solves per observation bought nothing usable, while the trained flow produced its $K = 32$ ensemble in 16.1 seconds and with *zero* forward solves.

E13 then finds the regime where the chain does converge. On Burgers at 256 points, recovering $u(0)$ from a noisy $u(T)$, two settings sit either side of the identifiability frontier. Four initial-condition modes at $T = 0.1$ is where pCN converges, at $\hat{R} \leq 1.013$ with effective sample sizes of 578 to 1581, and eight modes at $T = 0.4$ is where it does not, at $\hat{R}$ between 5.2 and 6.8.

Against the converged reference the result is not in our favour, and we report it anyway. pCN recovers $u(0)$ to 0.004–0.010 relative L^2 where the flow reaches 0.030–0.061, so a factor of five to ten, and the posterior of the flow is 6.6–11.1 $\times$ too wide. Where the data nearly determine the answer, the learned sampler does not become sharp enough, which is the opposite of the over-confidence Section 4.4 found on the hard problems. Past the frontier the picture inverts: there the two widths agree to within 12–39%, so the spread of the flow is about right exactly where the posterior is genuinely broad.

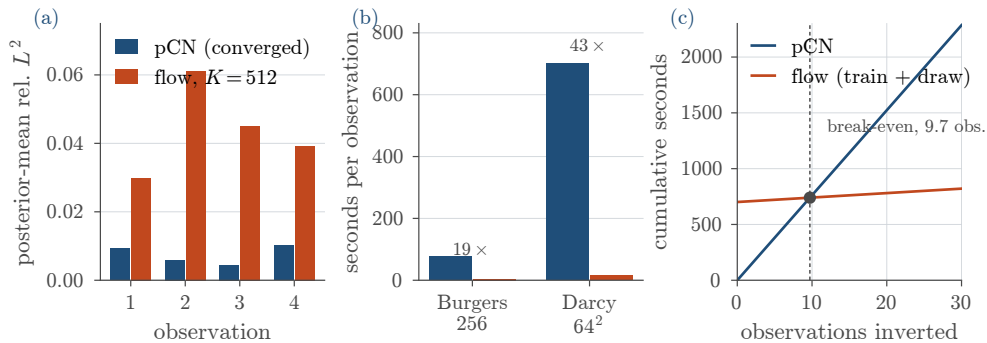


Fig. 14. E13, Burgers 256, identifiable regime, against E10’s Darcy reference. **(a)** Posterior-mean error where the chain converged: pCN is five to ten times more accurate. **(b)** Cost per observation on the same machine. **(c)** Cumulative cost against observations inverted. The surrogate’s training is a fixed offline cost and its per-observation cost is nearly flat, so the two lines cross at ten observations.

The break-even is ten observations. This is the number the reduced-order argument rests on, and it is small. Training costs 701 seconds, once, and each new observation costs the chain 76 seconds and the flow 4.0, so the cumulative curves cross at 9.7 observations, stable to within 1% across all four test observations. This gives a simple rule: below ten inversions we run the chain, above ten the surrogate has paid for itself

and every further observation is nearly free. It is the offline/online split of Section 2.2 as a measurement and not as an argument.

So what we can claim is not that the learned posterior is better. On the one problem where we can check it, it is worse. What we can claim is that it is *approximately right and amortized*, and that we have now measured what this trade costs.

Two things this does not establish. First, that vanilla pCN at this budget fails on Darcy does not mean Markov chain Monte Carlo is infeasible there. pCN takes the same step size in all 4096 directions while the observation constrains only a few hundred of them, so a dimension-reduced or gradient-informed sampler would very likely mix at a budget this one did not, and it would also be the right instrument for checking whether the $6.6\text{--}11.1\times$ over-dispersion measured on Burgers persists at 64×64 .

Second, we should record one withdrawn measurement. A companion forward-cost sweep on the same Burgers problem was run and then discarded. At $\nu = 10^{-2}$ the solution has decayed to numerical zero by $T = 6.4$, so the apparent $6.4\times$ surrogate advantage at the longest horizon was an artefact of the solver integrating 25,600 steps to produce nothing. At the one non-degenerate horizon the solver wins outright, which agrees with Section 4.3.

5 Conclusions

In this work, we developed a library where the main components of a flow-based PDE surrogate can be changed independently. We then used it to compare these choices on Poisson, Burgers and Darcy problems. The main results are the following.

Conditioning is important, but the training loss does not clearly show this. Removing the conditioner increases the median relative L^2 error by a factor of 17, while the training losses still look similar (Section 4.2). For this reason, we select models using sampled error instead of only the training loss.

Among the four tested backbones, the smallest model gives the best accuracy on both forward problems. On Burgers, it is $12.4\times$ more accurate than the largest alternatives, while attention gives no clear improvement beyond seed variation (Section 4.2). Therefore, increasing model size does not improve the result at this scale.

The sampling procedure is also important. The ODE solver and number of steps can be chosen after training while keeping the same model weights. At equal computational cost, first-order Euler performs better than fourth-order Runge–Kutta by up to $8.6\times$. Moreover, full numerical convergence is not always optimal. For our best model, sixteen Euler steps give 11% lower error than the adaptive solver (Section 4.3).

For the inverse problems, the evaluation metric changes the interpretation of the results. Proper scoring rules show about twice the effect of conditioning compared with the relative L^2 error of a single sample. It also detects calibration problems that relative L^2 does not show. In one case, longer training reduces the error but makes the model over-confident, with 90% coverage decreasing from 0.84 to 0.70 (Section 4.4).

The learned posterior also captures the identifiability structure of the Darcy problem. When we group the domain according to the local flux magnitude, the posterior width decreases by $3.9\times$ from the lowest to the highest flux decile. This agrees with the fact that κ is better identified where the flux contains more information.

Finally, we compared the learned sampler with a classical pCN chain. On the one problem where the reference chain converged, the learned sampler is five to ten times less accurate. However, the training cost is paid only once. For every new observation, the pCN chain costs about 76 seconds, while the flow costs about 4.0 seconds. Therefore, the total computational costs become comparable after about ten inverse problems (Section 4.6).

References

- [1] M. S. Albergo and E. Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations (ICLR)*, 2023.
- [2] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: Composable transformations of Python+NumPy programs, 2018. <http://github.com/google/jax>.
- [3] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [4] H. Chung, J. Kim, M. T. McCann, M. L. Klasky, and J. C. Ye. Diffusion posterior sampling for general noisy inverse problems. In *International Conference on Learning Representations (ICLR)*, 2023.
- [5] S. L. Cotter, G. O. Roberts, A. M. Stuart, and D. White. MCMC methods for functions: Modifying old algorithms to make them faster. *Statistical Science*, 28(3):424–446, 2013.
- [6] J. R. Dormand and P. J. Prince. A family of embedded Runge–Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.
- [7] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, K. Lacey, A. Goodwin, Y. Marek, and R. Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning (ICML)*, 2024.
- [8] V. Fortin, M. Abaza, F. Anctil, and R. Turcotte. Why should ensemble spread match the RMSE of the ensemble mean? *Journal of Hydrometeorology*, 15(4):1708–1713, 2014.
- [9] S. Fresca and A. Manzoni. POD-DL-ROM: Enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. *Computer Methods in Applied Mechanics and Engineering*, 388:114181, 2022.
- [10] T. Gneiting and A. E. Raftery. Strictly proper scoring rules, doi: 10.1214/13-STS421.

- prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.
- [11] W. Grathwohl, R. T. Q. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud. FFDJORD: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations (ICLR)*, 2019.
 - [12] T. M. Hamill. Interpretation of rank histograms for verifying ensemble forecasts. *Monthly Weather Review*, 129(3): 550–560, 2001.
 - [13] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
 - [14] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
 - [15] M. F. Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics — Simulation and Computation*, 18(3):1059–1076, 1989.
 - [16] F. Koehler, S. Niedermayr, R. Westermann, and N. Thuerey. APEBench: A benchmark for autoregressive neural emulators of PDEs. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
 - [17] G. Kohl, K. Um, and N. Thuerey. Benchmarking autoregressive conditional diffusion models for turbulent flow simulation. In *International Conference on Machine Learning (ICML)*, 2024.
 - [18] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhat-tacharya, A. Stuart, and A. Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *Journal of Machine Learning Research*, 24(89): 1–97, 2023.
 - [19] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhat-tacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.
 - [20] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
 - [21] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat. Flow matching guide and code. *arXiv preprint arXiv:2412.06264*, 2024.
 - [22] P. Lippe, B. S. Veeling, P. Perdikaris, R. E. Turner, and J. Brandstetter. PDE-Refiner: Achieving accurate long roll-outs with neural PDE solvers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
 - [23] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.
 - [24] I. Price, A. Sanchez-Gonzalez, F. Alet, T. R. Andersson, A. El-Kadi, D. Masters, T. Ewalds, J. Stott, S. Mohamed, P. Battaglia, R. Lam, and M. Willson. Probabilistic weather forecasting with machine learning. *Nature*, 637:84–90, 2025.
 - [25] A. Quarteroni, A. Manzoni, and F. Negri. *Reduced Basis Methods for Partial Differential Equations: An Introduction*, volume 92 of *UNITEX T*. Springer, 2016.
 - [26] D. J. Rezende and S. Mohamed. Variational inference with normalizing flows. In *International Conference on Machine Learning (ICML)*, pages 1530–1538, 2015.
 - [27] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.
 - [28] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.
 - [29] G. J. Székely and M. L. Rizzo. Energy statistics: A class of statistics based on distances. *Journal of Statistical Planning and Inference*, 143(8):1249–1272, 2013.
 - [30] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *Transactions on Machine Learning Research*, 2024.
 - [31] J. Wildberger, M. Dax, S. Buchholz, S. R. Green, J. H. Macke, and B. Schölkopf. Flow matching for scalable simulation-based inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.